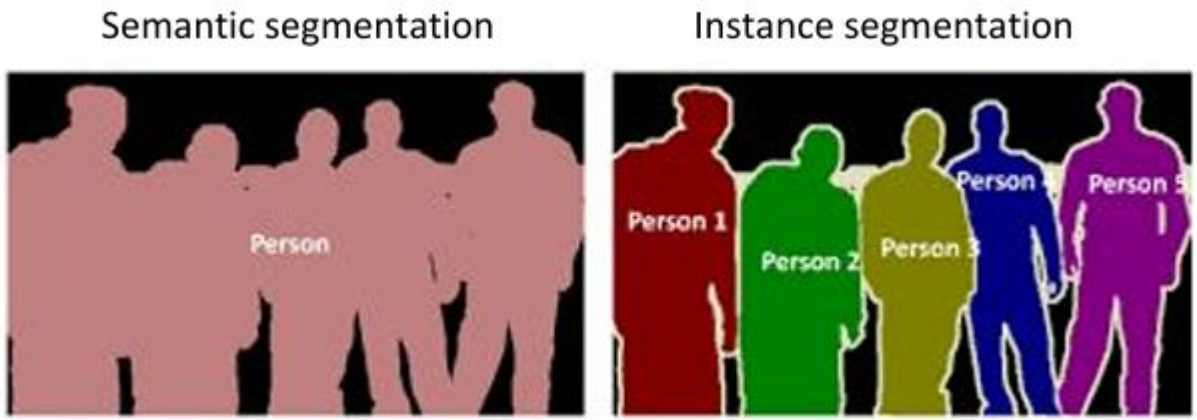
COMPUTER VISION · LECTURE

Image Segmentation

From pixels to masks: upsampling, dilated convolutions,
FCN, U-Net, DeepLab, Mask R-CNN and SegFormer

Shubham Jain



ROADMAP

Today's lecture

01 **What is segmentation?**

Semantic, instance, panoptic and friends

02 **Where it is useful**

Driving, medicine, earth observation, AR; datasets

03 **Dense prediction & core operators**

Upsampling, unpooling, transposed & dilated convolution

04 **Losses & metrics**

Pixel CE, focal, Dice; mIoU, Dice score, PQ

05 **CNN architectures**

FCN, SegNet, U-Net, Mask R-CNN, SETR

RECAP

Last time: boxes. Today: pixels.



A thin object at 45° fills only ~20% of its bounding box

- **Detection** answers what + roughly where: (class, x, y, w, h)
- Boxes are fine for counting and tracking, but poor at shape
- **Segmentation** answers what + exactly which pixels
  - Drivable area, lane markings, tumour boundaries, a person's silhouette
- Output size grows from a few numbers per object to one label per pixel

THINK Name two objects on an Indian road where a box would mislead a planner more than a mask would.

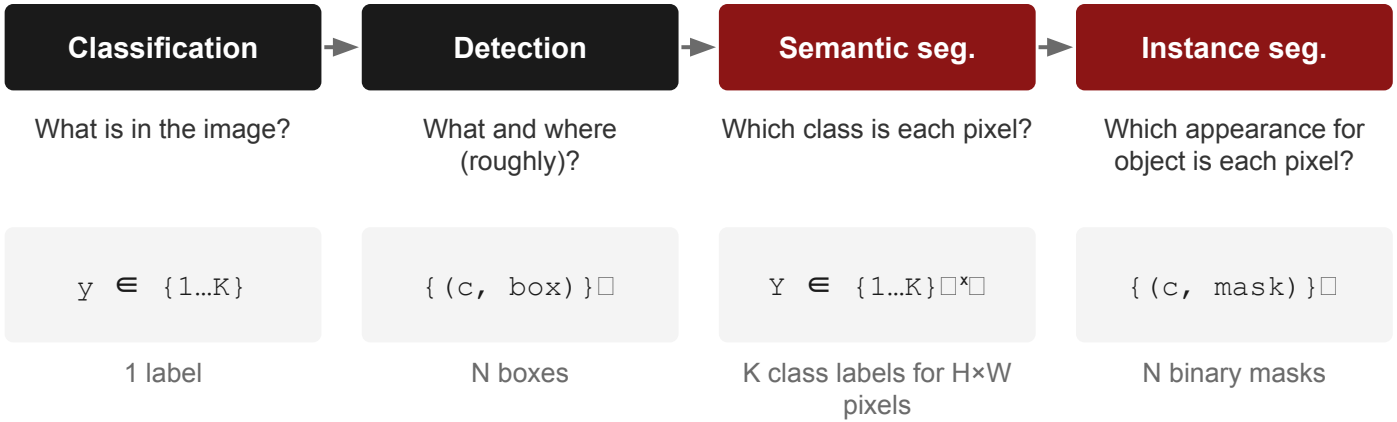
01

What is segmentation?

One task name, several very different output formats.

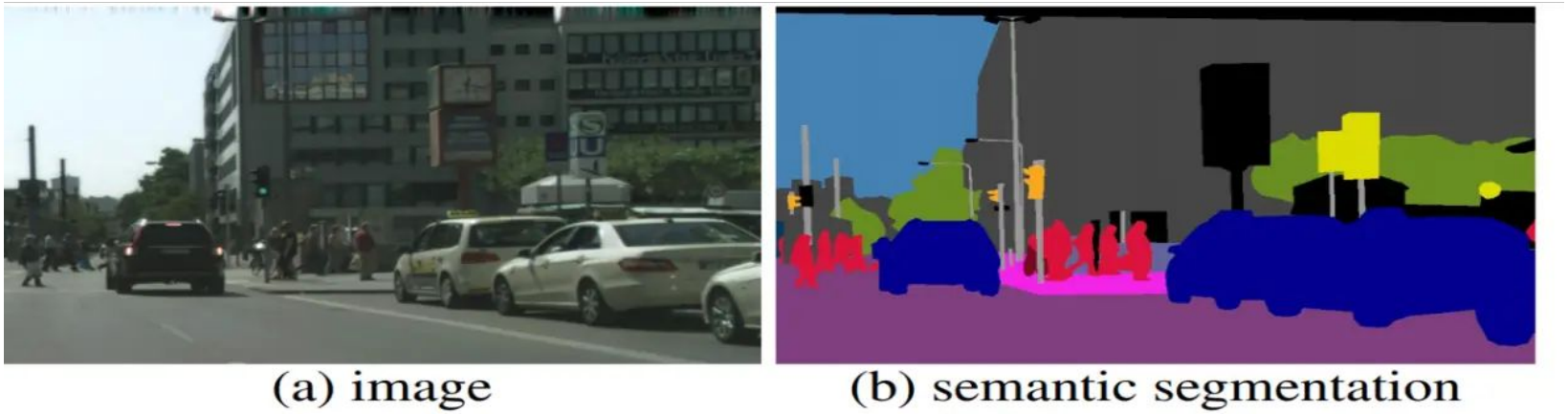


A ladder of increasingly dense outputs



KEY IDEA Moving right, output density increases and so does annotation cost. Choose the cheapest rung that answers your product question.

Semantic segmentation: a class for every pixel



- **Output:** H×W label map, one of K classes per pixel
- **Does not** separate instances: both cars get the same label
- **Covers everything:** road, sky, vegetation ("stuff") and cars, people ("things")

road
sky
vegetation
building
car
person

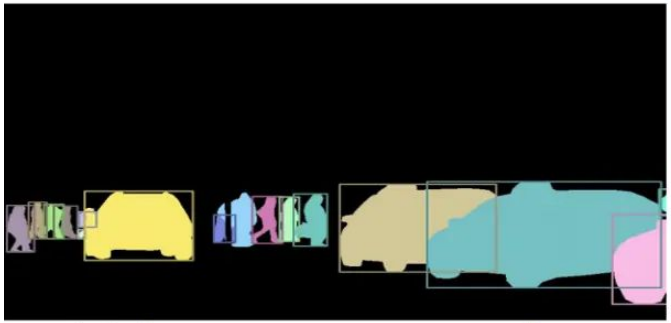
Instance segmentation: a mask for every object



(a) image



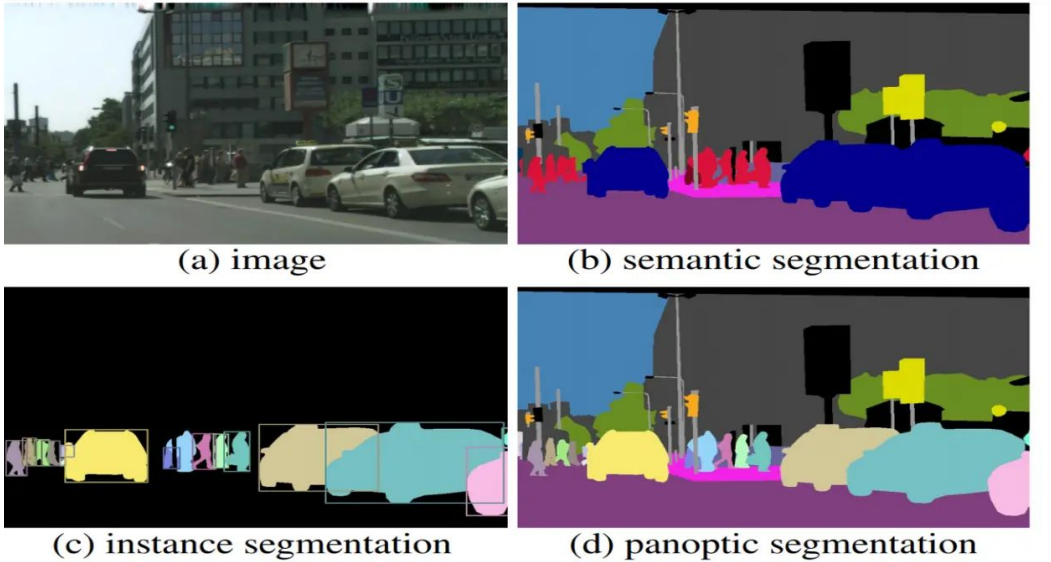
(b) semantic segmentation



(c) instance segmentation

- **Output:** a set {(class, score, binary mask)} — one entry per object
- **Separates** car #1 from car #2 → you can count, track, grasp
- **Ignores** amorphous background regions (road, sky)
- Typical models: Mask R-CNN, SOLO, Mask2Former

Panoptic segmentation: semantic + instance, no overlaps



- **Every pixel** gets (class, instance-id); stuff pixels share one id per class
- **Non-overlapping:** each pixel belongs to exactly one segment
- Introduced by Kirillov et al., CVPR 2019, with the Panoptic Quality (PQ) metric
- The natural output for a full scene understanding stack (e.g. AV perception)

"Stuff" vs "things"

THINGS

- Countable objects with a well-defined shape
- car, person, bicycle, traffic sign, cell nucleus
- Instance IDs make sense

STUFF

- Amorphous regions defined by texture or material
- road, sky, grass, water, wall
- "How many skies?" is not a meaningful question

Two car instances (blue, orange) on stuff (road, sky)

THINK Is a "crowd of pedestrians 200 m away" stuff or things? Datasets disagree — COCO has an iscrowd flag for exactly this reason.

KINDS OF SEGMENTATION

The wider family of segmentation tasks

Binary / foreground

Object vs background. Salient-object detection, image matting (soft alpha), portrait mode.

Part segmentation

Parts of an object: human parsing (hair, face, arms), car parts for damage assessment.

Video object seg.

Track given masks through time. Benchmarks: DAVIS, YouTube-VOS.

3D / point clouds

Per-point labels on LiDAR scans. SemanticKITTI, nuScenes-lidarseg; PointNet++, Cylinder3D.

Referring / open-vocab

Segment what a text phrase describes: "the man in the red shirt". Not limited to a fixed class list.

Promptable / interactive

Click, box or text prompt → mask. SAM, SAM 2, SAM 3. The default tool for annotation today.

Side by side

Task	Label per pixel	Separates instances?	Covers stuff?	Metric	Example models
Semantic	class	No	Yes	mIoU	FCN, U-Net
Instance	(class, id) for things	Yes	No	mask AP	Mask R-CNN
Panoptic	(class, id) for all	Yes	Yes	PQ	Panoptic FPN,

THINK A warehouse robot must pick individual boxes off a pallet. Which row do you need, and why is the semantic row not enough?

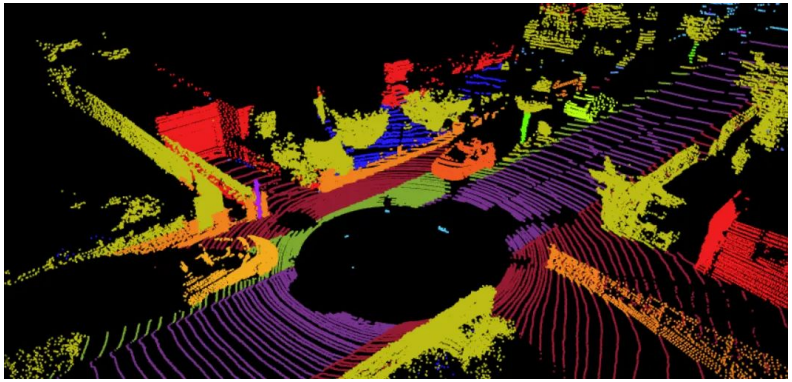
02

Where is segmentation useful?

Anywhere shape, area or exact boundaries matter more than a rough location.



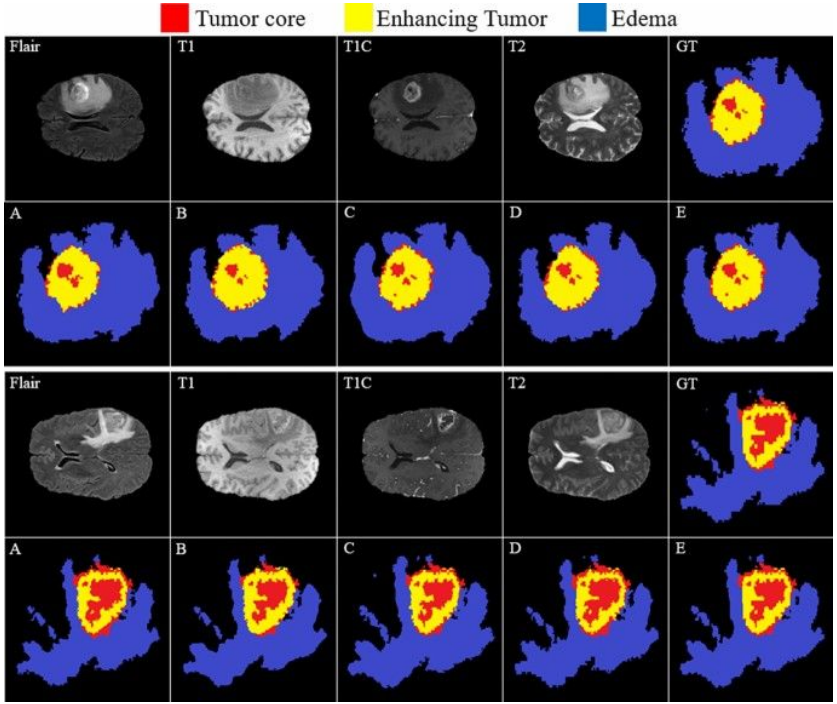
Autonomous driving and ADAS





- **Drivable space / free space** for path planning
- **Lane markings, curbs, road edges** — thin structures boxes cannot describe
- **Pedestrians and riders** with exact extents for time-to-collision
- **BEV semantic maps** fused from multiple cameras
- **LiDAR point-cloud segmentation** (road, vehicle, vegetation per point)
- Constraint: tens of milliseconds on an automotive SoC

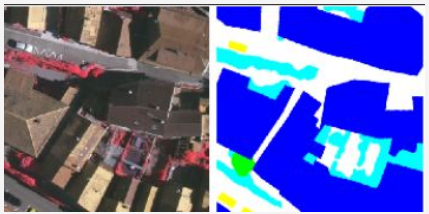
Medical imaging



- **Tumour and lesion segmentation** (brain MRI, liver CT, skin lesions)
- **Organ-at-risk contouring** for radiotherapy planning
- **Cell and nucleus segmentation** in histopathology and microscopy
- **Retinal vessels, cardiac chambers**, polyps in endoscopy
- Often 3D volumes; few labelled scans; severe class imbalance
- Benchmarks: BraTS, ISIC, Medical Segmentation Decathlon

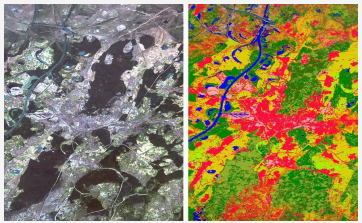
APPLICATIONS

Earth observation, agriculture and industry



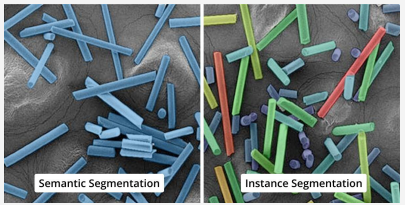
Remote sensing

Land cover, building footprints, road networks, flood and deforestation mapping from satellite tiles.



Precision agriculture

Crop vs weed masks for targeted spraying; leaf-disease area; fruit counting.



Industrial inspection

Crack, scratch and corrosion masks; measure defect area, not just presence.

KEY IDEA When the answer to your question is an area, a length or a boundary, you need segmentation, not detection.

Robotics, AR and everyday photo apps

- **Robotic grasping:** instance masks give object extent and pose cues
- **Mobile robots:** traversable-terrain segmentation for navigation
- **Portrait mode / video-call blur:** person vs background, per frame, on-device
- **Photo editing:** "select subject", object removal and inpainting
- **AR:** occlusion — virtual objects must go behind real people
- **Data labelling:** SAM-style tools cut mask annotation time sharply



APPLICATIONS

Benchmark datasets you will meet

Dataset	Domain	Classes	Size (commonly used)	Task
PASCAL VOC 2012	Everyday objects	20 + background	1.5K train / 1.4K val (10.6K with SBD aug.)	Semantic
Cityscapes	Urban driving, 50 cities	19 evaluated	5K fine (2975/500/1525) + 20K coarse, 2048×1024	Sem. / inst. / pan.
ADE20K	Diverse scenes	150	20K train / 2K val	Semantic
COCO	Everyday objects	80 things (+53 stuff)	118K train / 5K val	Inst. / panoptic
SemanticKITTI	LiDAR scans	19 evaluated	22 sequences	Point-cloud sem.

Also: Medical Segmentation Decathlon, BraTS, ISIC (medical); SA-1B (SAM's 11M images / 1.1B masks).

APPLICATIONS

Why labels are the bottleneck

~1.5 h

to finely annotate one Cityscapes image

~79 s

per object mask in COCO

1.1 B

masks in SA-1B, produced with a model in the loop

- **Weak labels:** boxes, points, scribbles, image-level tags
- **Semi-supervised** training on unlabeled frames; **synthetic data** from simulators
- **Model-in-the-loop** annotation: a promptable model proposes, a human corrects

THINK Your budget is 200 annotator-hours. Would you rather have 130 fine Cityscapes-style images or 10,000 box-labelled ones?

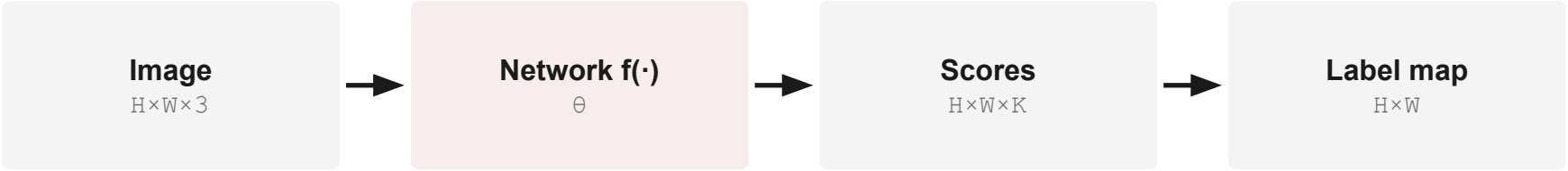
03

Dense prediction & core operators

How do we get a prediction for every pixel without melting the GPU?



Segmentation = classification at every pixel



softmax + argmax over K

$$p(i,j,k) = \frac{\exp(s_{i,j,k})}{\sum_{k'=1}^{C} \exp(s_{i,j,k'})}$$

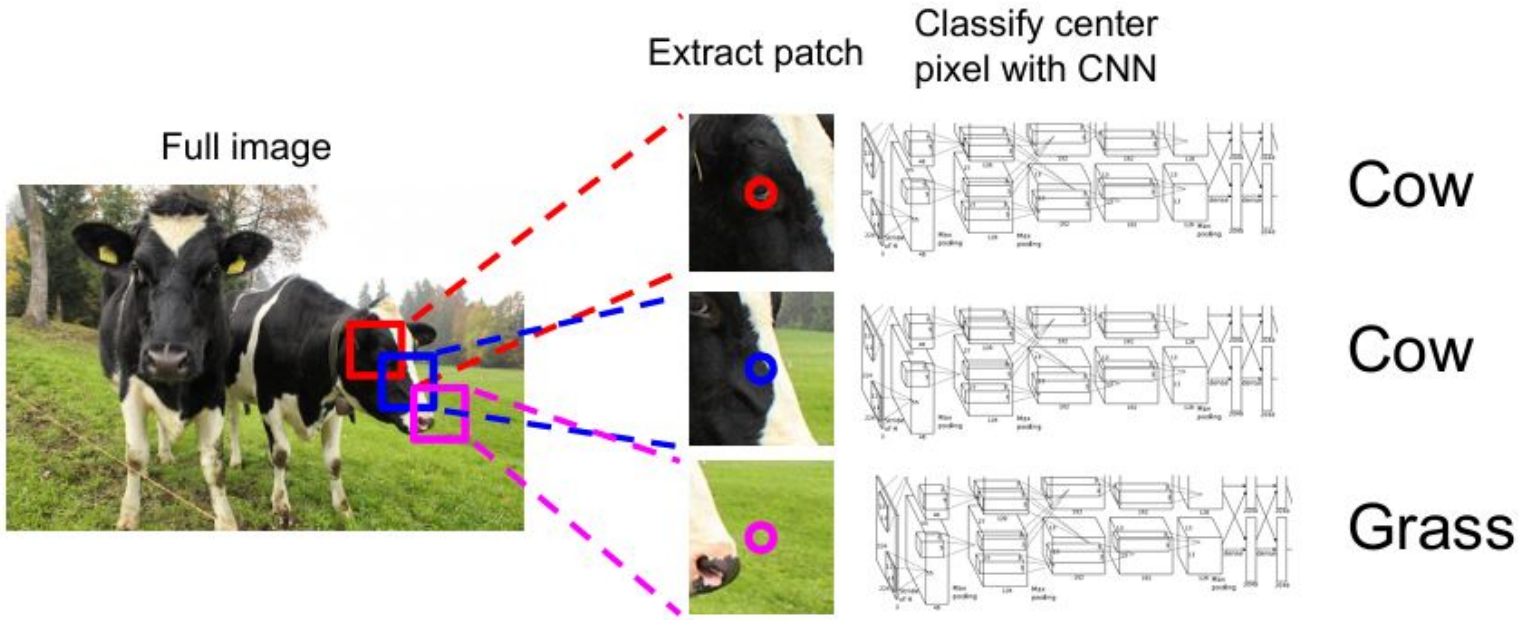
$$\hat{y}_{i,j} = \underset{k \in \{1,\ldots,C\}}{\arg\max}\ p(i,j,k)$$

Where:

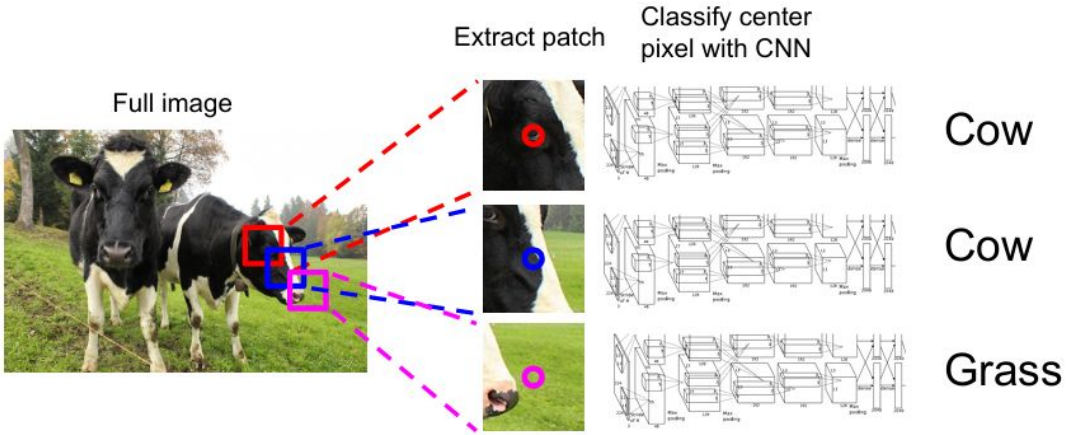
Term	Explanation
$p(i,j,k)$	Probability that pixel (i,j) belongs to class k.
$s_{i,j,k}$	Logit (raw score) predicted by the model for class k at pixel (i,j).
$\exp(\cdot)$	Exponential function, e^x, used to convert logits into positive values.
i,j	Spatial coordinates of the pixel (row and column).
k	Current class being evaluated.
k'	Index used to iterate over all classes in the denominator.
C	Total number of segmentation classes.

- Same network, same loss as classification — applied H×W times
- Cityscapes: 1024×2048 ≈ 2.1 M pixel predictions per image
- Hard part: every prediction needs both **local detail** and **global context**

Naive idea #1: classify a patch around every pixel

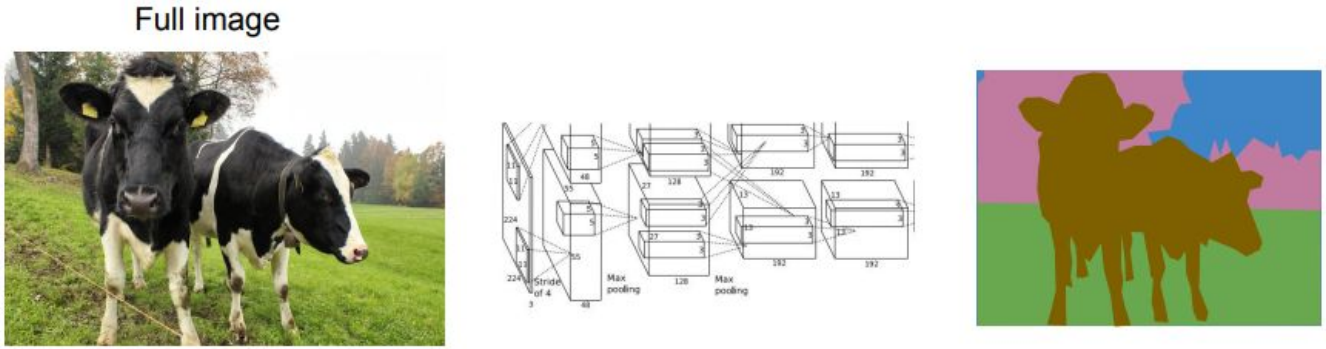


Naive idea #1: classify a patch around every pixel



- **Cost:** one forward pass per pixel → ~2.1 M passes for a Cityscapes image
- **Redundant:** adjacent patches share almost all pixels, features recomputed each time
- **Fixed context:** the patch size caps how much the model can see

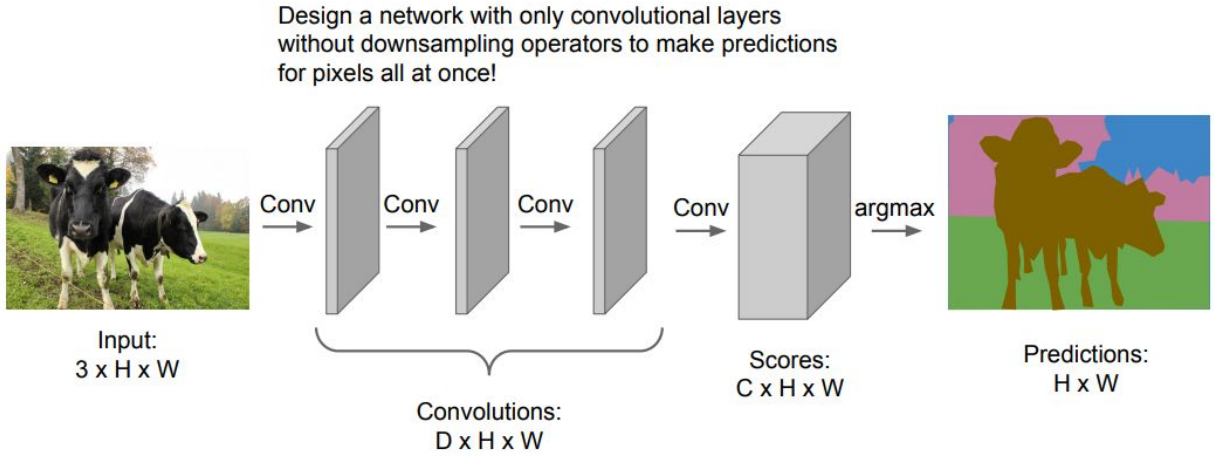
Naive idea #2: stack convolutions at full resolution



An intuitive idea: encode the entire image with conv net, and do semantic segmentation on top.

- **Problem: classification architectures often reduce feature spatial sizes to go deeper, but semantic segmentation requires the output size to be the same as input size**

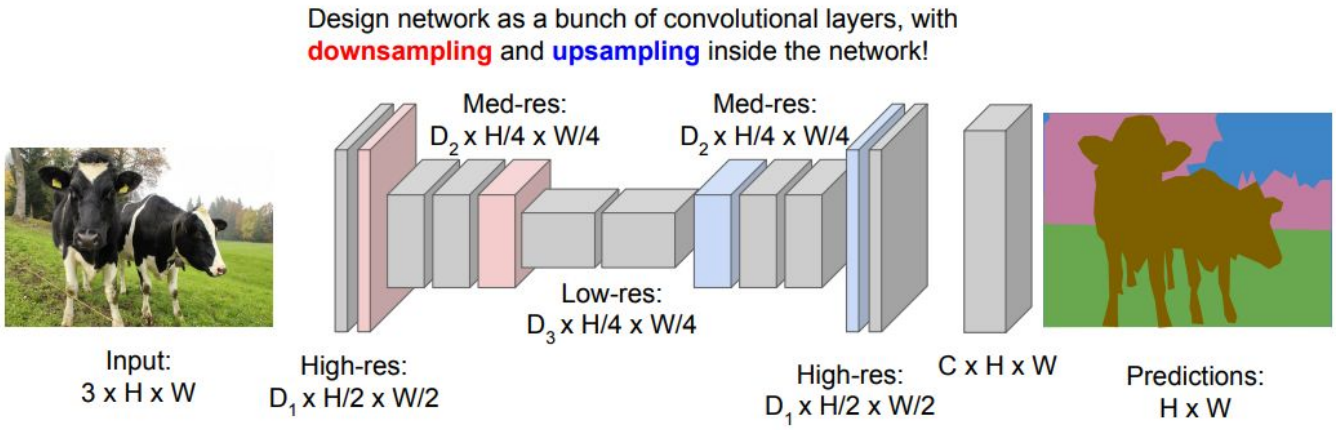
Naive idea #2: stack convolutions at full resolution



Problems

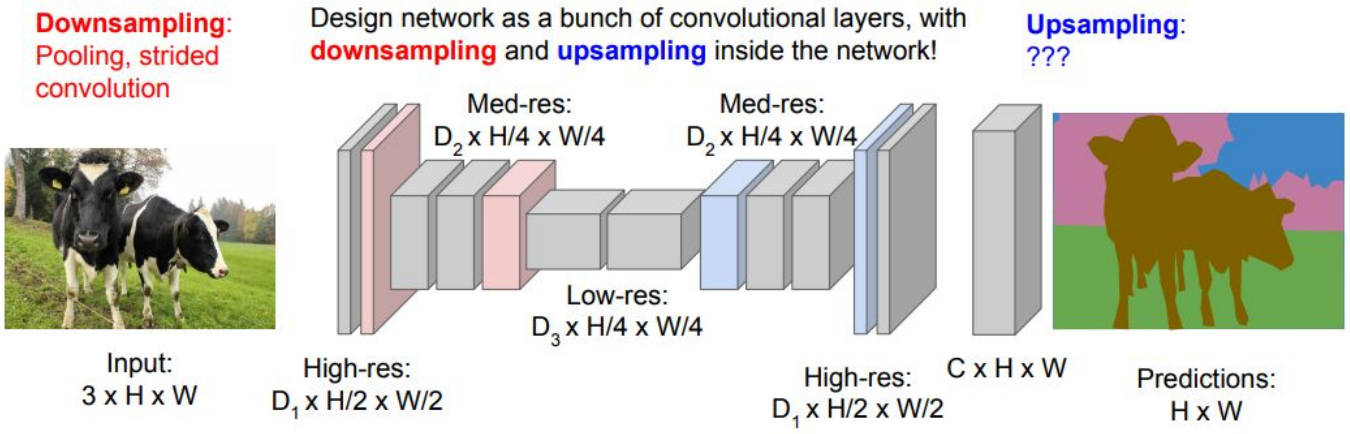
- **Memory:** activations scale with H×W×C at every layer
- **Context:** to see 1,000 px you would need ~500 layers of 3×3
- We need to see far **and** stay affordable → reduce resolution somewhere

The fix: downsample, then upsample (encoder–decoder)



- **Encoder** = a classification backbone (VGG, ResNet, MiT): cheap, large receptive field
- **Bottleneck** holds semantics ("what") at low resolution
- **Decoder** restores resolution ("where")
- Output stride (OS) = input size / feature size, e.g. OS 32 → 1024×2048 becomes 32×64

The fix: downsample, then upsample (encoder–decoder)



- **Encoder** = a classification backbone (VGG, ResNet, MiT): cheap, large receptive field
- **Bottleneck** holds semantics ("what") at low resolution
- **Decoder** restores resolution ("where")
- Output stride (OS) = input size / feature size, e.g. OS 32 → 1024×2048 becomes 32×64

Nearest neighbour and bilinear, by hand

Input 2×2

```
1   2
3   4
```

Nearest ×2 → 4×4

```
1   1   2   2
1   1   2   2
3   3   4   4
3   3   4   4
```

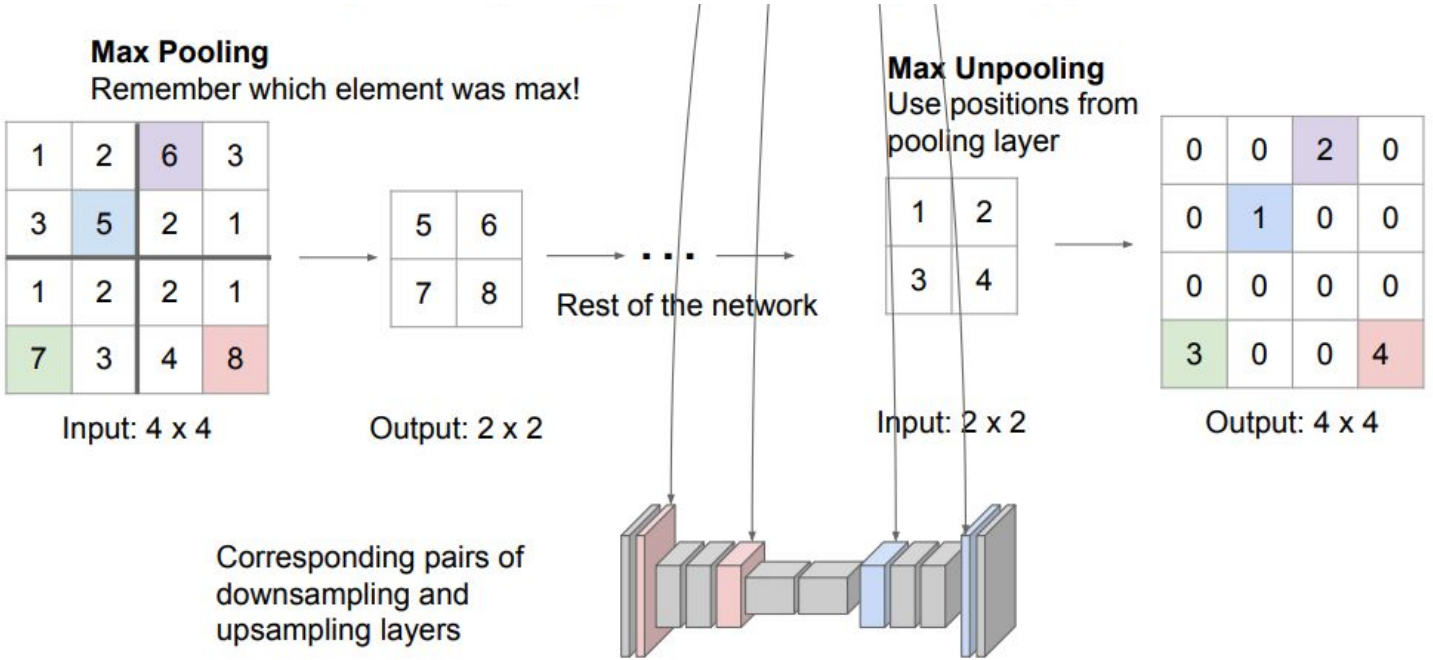
Bilinear, align_corners=True → 3×3

```
1.0   1.5   2.0
2.0   2.5   3.0
3.0   3.5   4.0
```

$$f(x,y) \approx (1-a)(1-b)\cdot f00 + a(1-b)\cdot f10 + (1-a)b\cdot f01 + ab\cdot f11$$

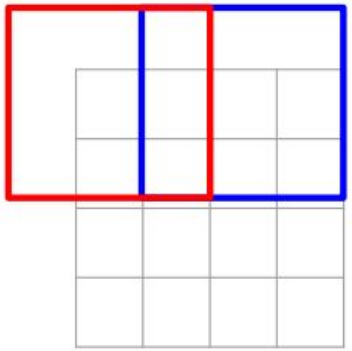
- **Nearest:** fast, blocky edges, no new values
- **Bilinear:** smooth, can blur boundaries; the standard final step to full resolution
- Gotcha: align_corners=True vs False gives different grids — keep training and export consistent

Unpooling: bed of nails and max unpooling



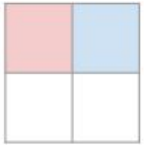
Transposed convolution: learnable upsampling

Recall: Normal 3 x 3 convolution, stride 2 pad 1



Input: 4 x 4

Dot product between filter and input



Output: 2 x 2

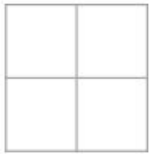
Filter moves 2 pixels in the input for every one pixel in the output

Stride gives ratio between movement in input and output

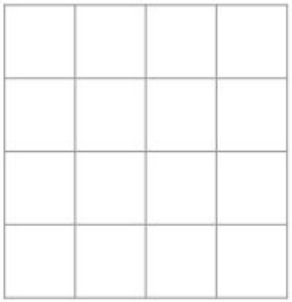
We can interpret strided convolution as “learnable downsampling”.

Transposed convolution: learnable upsampling

3 x 3 **transposed** convolution, stride 2 pad 1

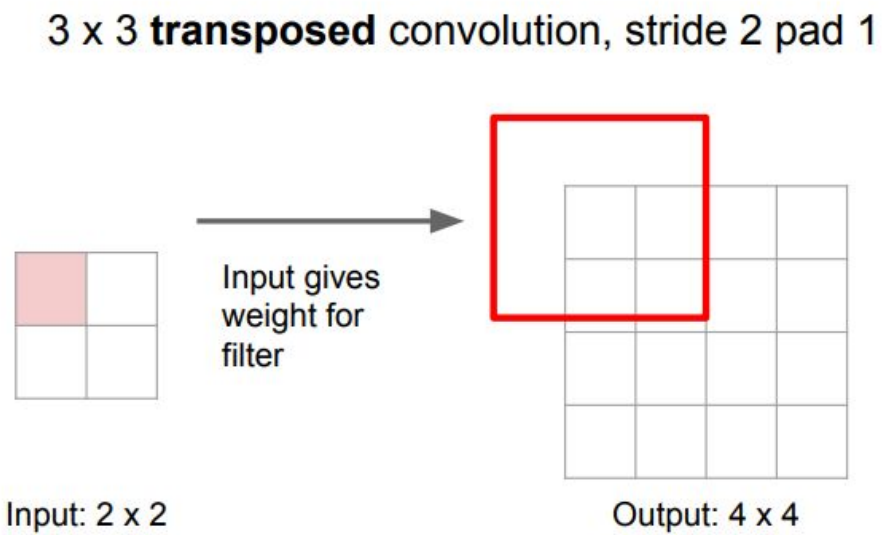


Input: 2 x 2

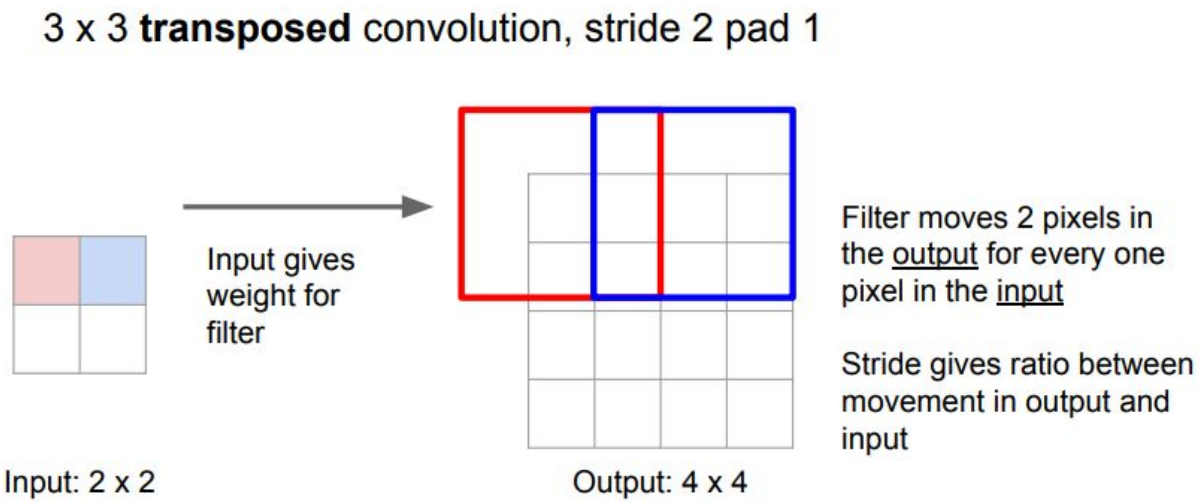


Output: 4 x 4

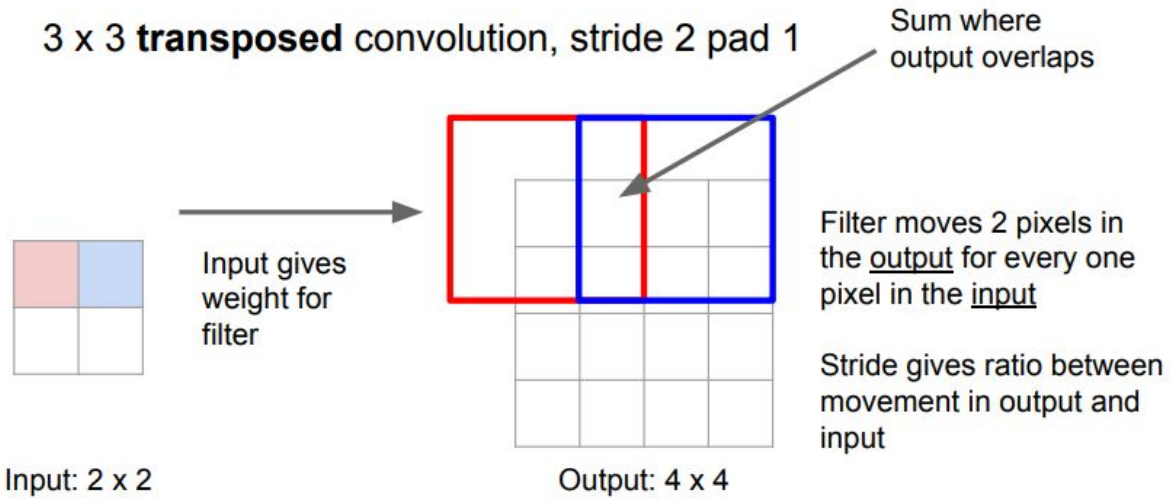
Transposed convolution: learnable upsampling



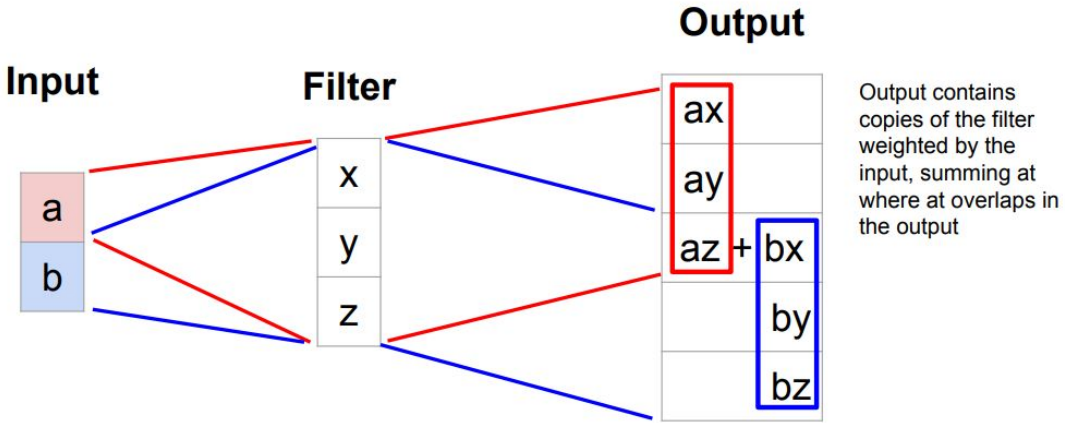
Transposed convolution: learnable upsampling



Transposed convolution: learnable upsampling



Transposed convolution: learnable upsampling



- Each input value **multiplies the whole k×k kernel** and writes it to the output
- The **stride sets the step on the output grid** (stride 2 → roughly 2× larger)
- **Overlapping stamps are summed**
- Weights are learned like any conv; often initialised to bilinear (FCN did this)
- Other names: fractionally-strided conv, up-convolution, "deconvolution" (misleading)

Output size in 2D

$$o = (i - 1) \cdot s - 2p + k + \text{output_padding}$$

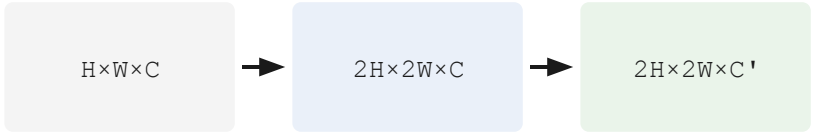
i	k	s	p	out_pad	o	Comment
4	3	1	0	0	6	stride 1 still grows by k − 1
4	2	2	0	0	8	U-Net up-conv: no overlap
4	4	2	1	0	8	clean ×2, k divisible by s
4	3	2	1	1	8	×2, but uneven overlap
16	4	2	1	0	32	a typical decoder step

```
up = nn.ConvTranspose2d(
  in_channels=256,
  out_channels=128,
  kernel_size=4,
  stride=2,
  padding=1)
# 16×16 → 32×32
```

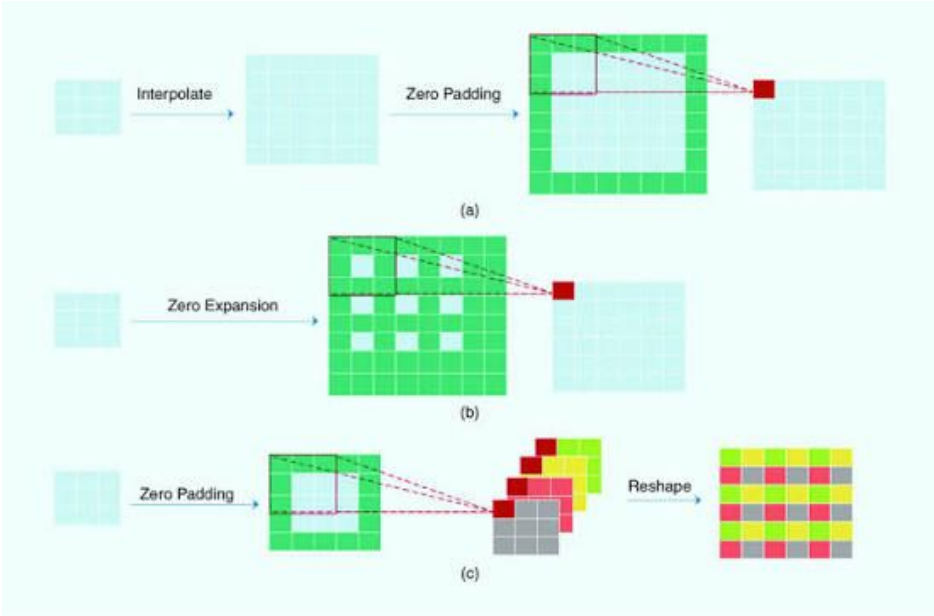
Parameters: k·k·C_in·C_out (+C_out bias) — same as a normal conv

Artifact-free learnable upsampling

Resize + convolution



- No uneven overlap by construction
- Default in many modern decoders
- Cost: conv runs at the higher resolution



Operator cheat-sheet

Operator	Changes size?	Learnable?	Watch out for	Seen in
Nearest / bilinear	Up	No	Blocky / blurry edges	FPN, DeepLab, SegFormer
Max unpooling	Up	No (needs indices)	Sparse output	SegNet
Transposed conv	Up	Yes: $k^2 \cdot Cin \cdot Cout$	Checkerboard if $k \% s \neq 0$	FCN, U-Net
Resize + conv	Up	Yes: $k^2 \cdot Cin \cdot Cout$	Conv at high res costs more	Modern decoders
Pixel shuffle	Up ×r	Yes: $k^2 \cdot Cin \cdot Cout \cdot r^2$	Channel count × r^2	ESPCN, super-res

Exercise: sizes and receptive fields

1. ConvTranspose2d(k = 4, s = 2, p = 1) on a 16×16 map. Output size?
2. Three stacked 3×3 convs with dilation 1, 2, 4. Receptive field?
3. Encoder with output stride 32 on a 512×1024 image. Feature map size? And at OS 8?
4. Why did FCN initialise its transposed convs to bilinear interpolation?
5. 3×3 conv, dilation 6, padding 6, stride 1 on 64×64. Effective kernel size and output size?

Work in pairs for 5 minutes. Answers in the speaker notes.

04

Losses & metrics

How to train a per-pixel classifier, and how to know if it is any good.



Pixel-wise cross-entropy

$$L = -\sum_{k=1}^{K} y_k \log(p_k)$$

- Average over **all N labelled pixels** in the batch
- Void / unlabelled pixels skipped with **ignore_index = 255**
- Computed on logits upsampled to label size (or labels downsampled)
- Simple and stable — the default starting point

```
loss = F.cross_entropy(logits, target,
                       ignore_index=255)
```



loss at this pixel = −ln 0.70 = 0.357

Class imbalance: weighted CE and focal loss

- **Problem:** road, sky, building dominate pixel counts; poles, lights, bikes are tiny
- **Weighted CE:** w_k ∝ 1 / freq_k (or median-frequency balancing, as in SegNet)
- **Focal loss:** down-weights easy pixels so hard ones drive the gradient

$$FocalLoss = -\sum_{i=1}^{i=n}(i - p_i)^{\gamma} log_b(p_i)$$

THINK Your model reaches 95% pixel accuracy but never predicts "traffic light". Which of these fixes would you try first, and why?

Dice loss: optimise overlap directly

$$\text{Dice coefficient} = \frac{\text{2x area of overlapped (green)}}{\text{total area (green)}} = \frac{2 \times}{\text{predicted}} \quad \frac{\text{ground truth}}{\text{predicted}} \quad \frac{}{\text{ground truth}}$$

Dice Loss = 1 - Dice Coefficient



Lesion < 1% of pixels: predicting 'all background' gets > 99% accuracy but Dice = 0

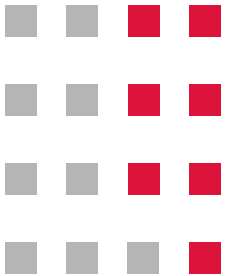
Metrics: pixel accuracy, IoU, mIoU, Dice

Pixel accuracy	IoU (per class)	mIoU	Dice / F1
`correct / total`	`TP / (TP + FP + FN)`	`mean of per-class IoU`	`2TP / (2TP + FP + FN)`
Misleading under imbalance	Standard for segmentation	Rare classes count equally	= 2·IoU / (1 + IoU)

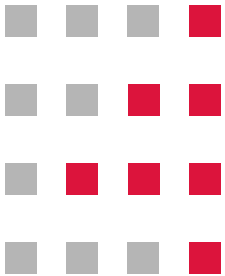
- **Accumulate** TP / FP / FN over the whole dataset per class, then divide (not a mean of per-image IoUs)
- Report **per-class IoU** too — a good mIoU can hide a class that is always wrong
- Instance segmentation uses **mask AP** (COCO-style, IoU on masks); panoptic uses **PQ**

Worked example: 4×4, two classes

Ground truth



Prediction



	|GT|	|Pred|	∩	∪	IoU
class 0 (grey)	9	9	8	10	0.80
class 1 (red)	7	7	6	8	0.75

0.775
mIoU

0.875
Pixel acc.

0.857
Dice (cls 1)

THINK Add a third class that appears in GT once and is never predicted. What happens to mIoU vs pixel accuracy?

Panoptic Quality (PQ)

$$\mathrm{PQ} = \underbrace{\frac{\sum_{(p,g)\in TP} \mathrm{IoU}(p,g)}{|TP|}}_{\text{segmentation quality (SQ)}} \times \underbrace{\frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}}_{\text{recognition quality (RQ)}}.$$

```
PQ = SQ (mean IoU of matched)  ×  RQ ( |TP| / (|TP| + ½|FP| + ½|FN|) )
```

- **Match** if IoU > 0.5 → each GT matches at most one prediction
- **SQ:** how well matched segments are drawn
- **RQ:** F1 of detection — did we find the right segments?
- Computed per class, averaged over classes; reported for things and stuff separately

Example

```
TP: IoU 0.8, 0.6   FP: 1   FN: 1
SQ = (0.8 + 0.6) / 2   = 0.70
RQ = 2 / (2 + 0.5 + 0.5) = 0.667
PQ = 0.70 × 0.667      = 0.467
```

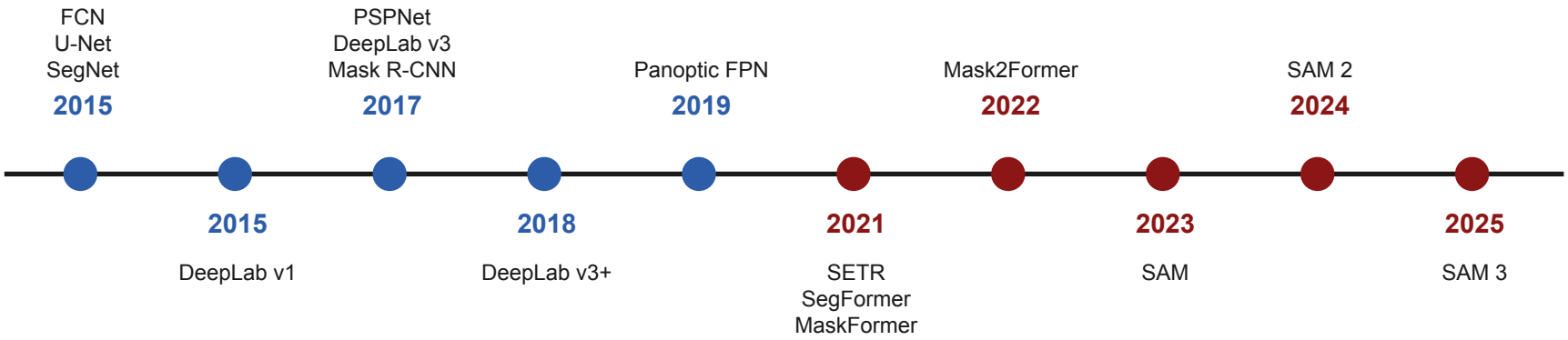
05

CNN architectures

FCN → SegNet → U-Net → DeepLab → PSPNet → Mask R-CNN

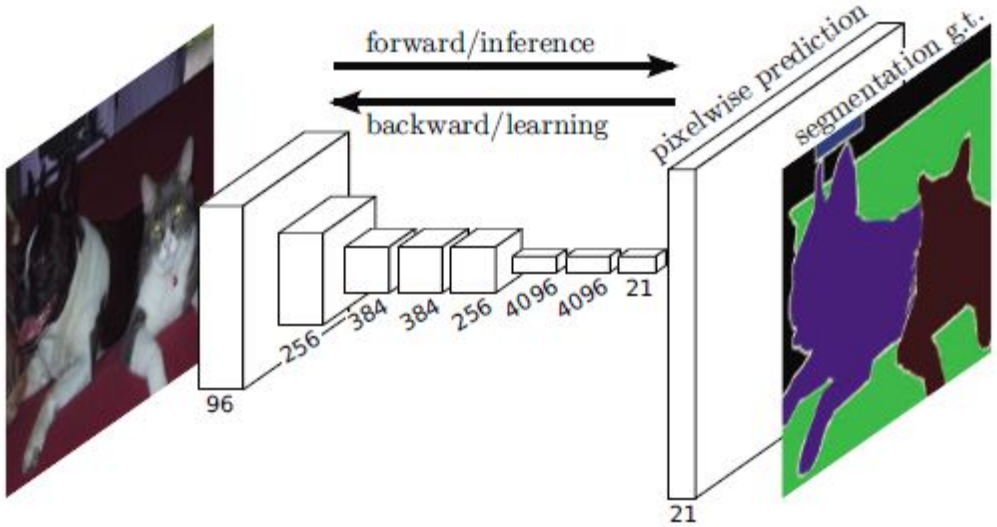


A decade of segmentation networks



Blue: CNN era · Red: transformer / foundation-model era

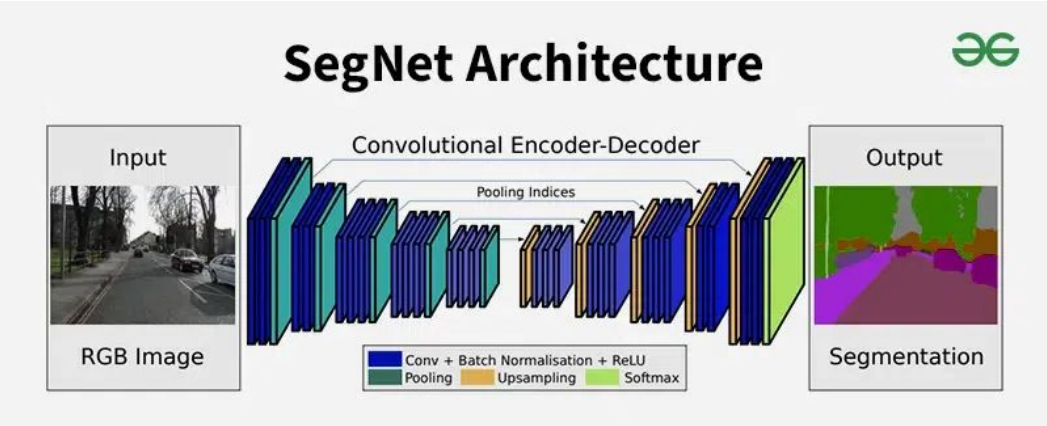
FCN



KEY IDEA

- Input: 512 × 512 × 3
- Number of classes: 21
- Output: 512 × 512 × 21 logits (in a typical FCN-8s-style model).

SegNet: a symmetric encoder–decoder with pooling indices



Encoder (VGG-16 convs)

Decoder (unpool + conv)

- **Max unpooling** with indices from the matching encoder layer
- Trainable convs densify the sparse unpooled map
- **Memory-light:** stores indices, not feature maps
- Median-frequency class balancing in the loss
- Targeted at road scenes and real-time use

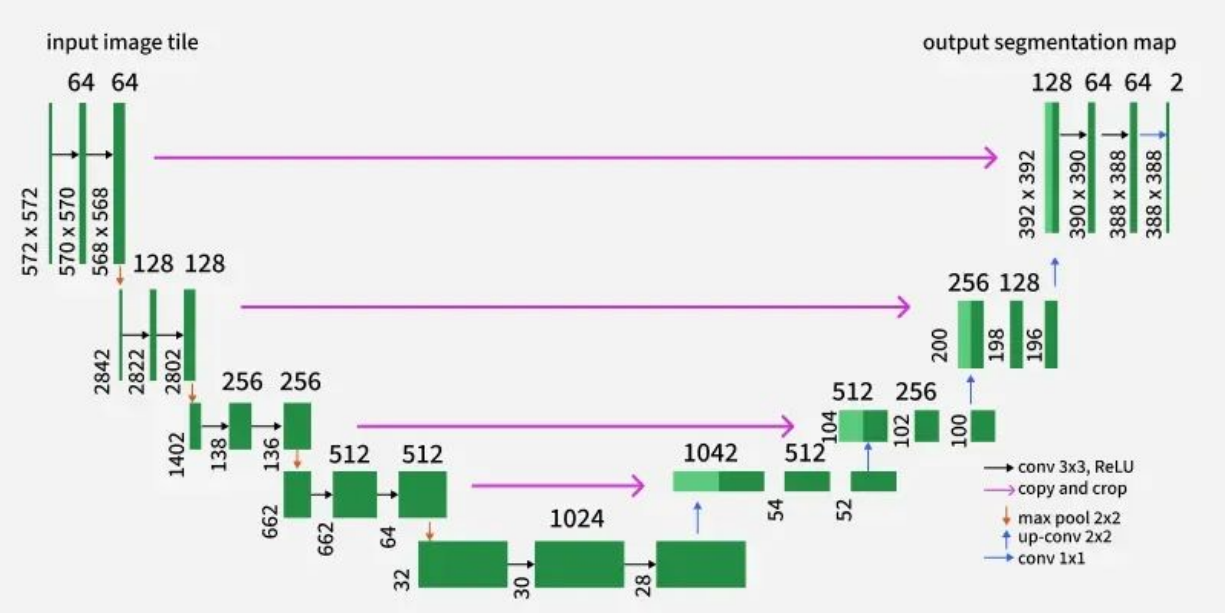
THINK What detail can indices recover that a bilinear upsample cannot? What can they not recover?

U-Net: the architecture to remember

572×572 in

copy & crop (concatenate)

388×388 out



- **Down:** 2× (3×3 conv, ReLU), then 2×2 max-pool; channels double
- **Up:** 2×2 up-conv halves channels; **concatenate** encoder features; 2× 3×3 conv
- **Skips concatenate** (U-Net) rather than add (FCN)
- **1×1 conv** at the end → class scores
- Symmetric "U" gives the name

Why U-Net worked with ~30 training images

~30

training images (ISBI 2012 EM)

572→388

valid (unpadded) convs shrink output

< 1 s

for a 512×512 image on a 2015 GPU

- **Skip concatenation** gives the decoder full-resolution detail at every level
- **Elastic deformation** augmentation — realistic for tissue
- **Weighted loss** that up-weights thin gaps between touching cells → separates instances
- **Overlap-tile** with mirrored borders → segment images of any size
- Won the ISBI 2015 cell tracking challenge by a large margin

The U-Net family is everywhere

3D U-Net
2016

Volumetric convs for CT / MRI stacks

Attention U-Net
2018

Attention gates filter skip features

U-Net++
2018

Nested, dense skip pathways

nnU-Net
2021

Self-configuring pipeline; tough baseline in medical imaging

TransUNet
2021

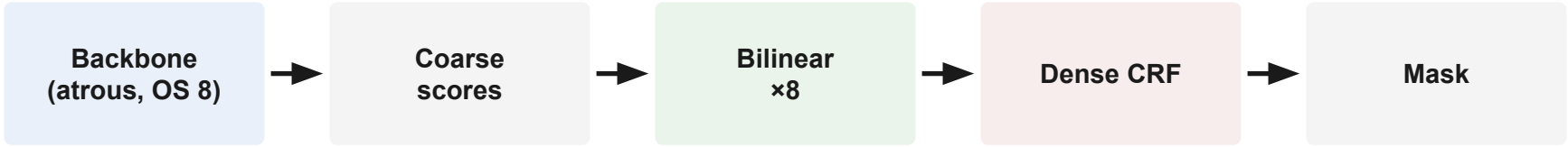
Transformer in the bottleneck for global context

Diffusion U-Nets
2022+

The denoiser in Stable Diffusion and many image generators

Takeaway: U-Net is a pattern (multi-scale encoder, symmetric decoder, concat skips), not a fixed network.

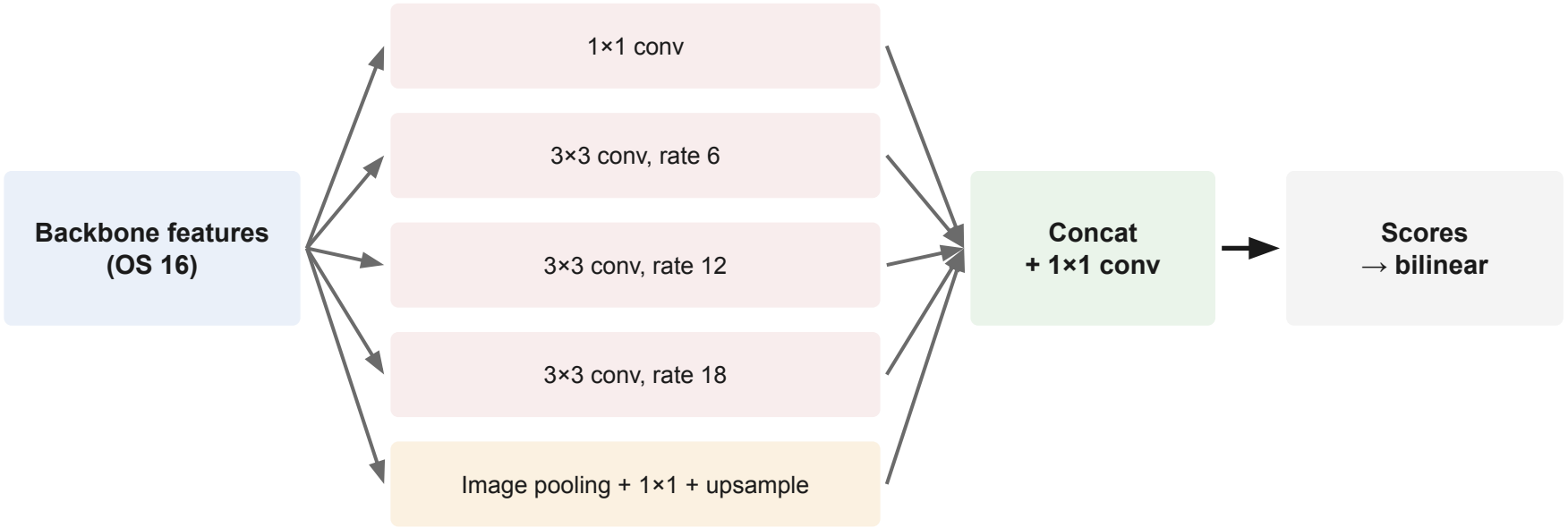
DeepLab v1 / v2: atrous convolution + CRF



- **Atrous backbone:** keep 1/8 resolution, pretrained context preserved
- **Fully connected CRF** post-processing snaps labels to image edges
- **v2 → ASPP:** parallel atrous branches with rates 6, 12, 18, 24 for multi-scale objects
- No learned decoder — upsampling is plain bilinear
- **v3 removed the CRF**: better features made it unnecessary

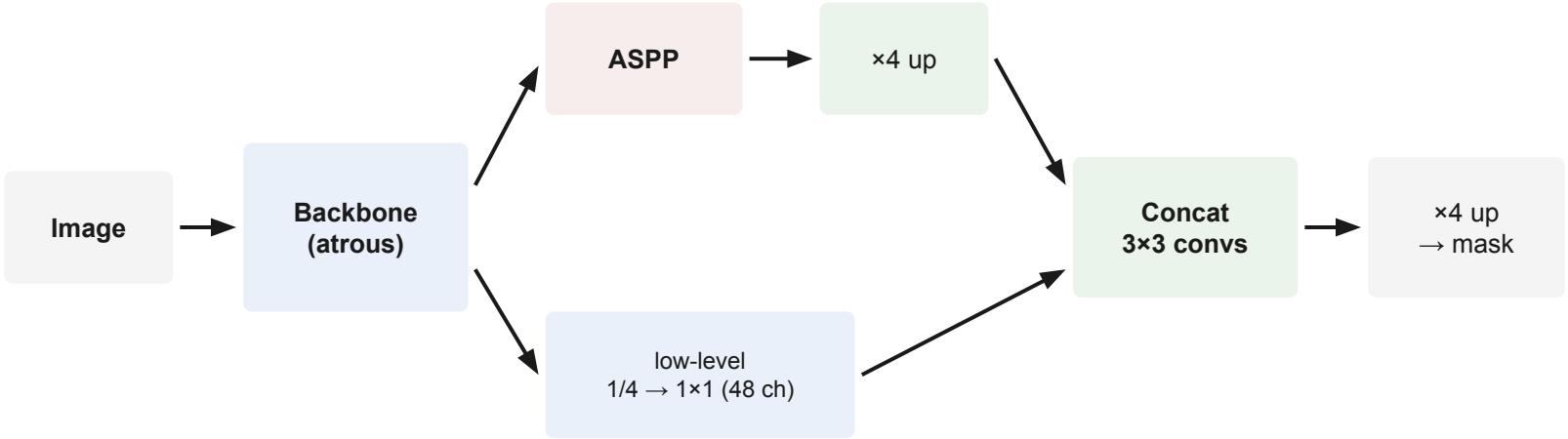
THINK Why might a CRF that uses pixel colour help at boundaries but hurt on a black car on dark asphalt?

ASPP: Atrous Spatial Pyramid Pooling (DeepLab v3)



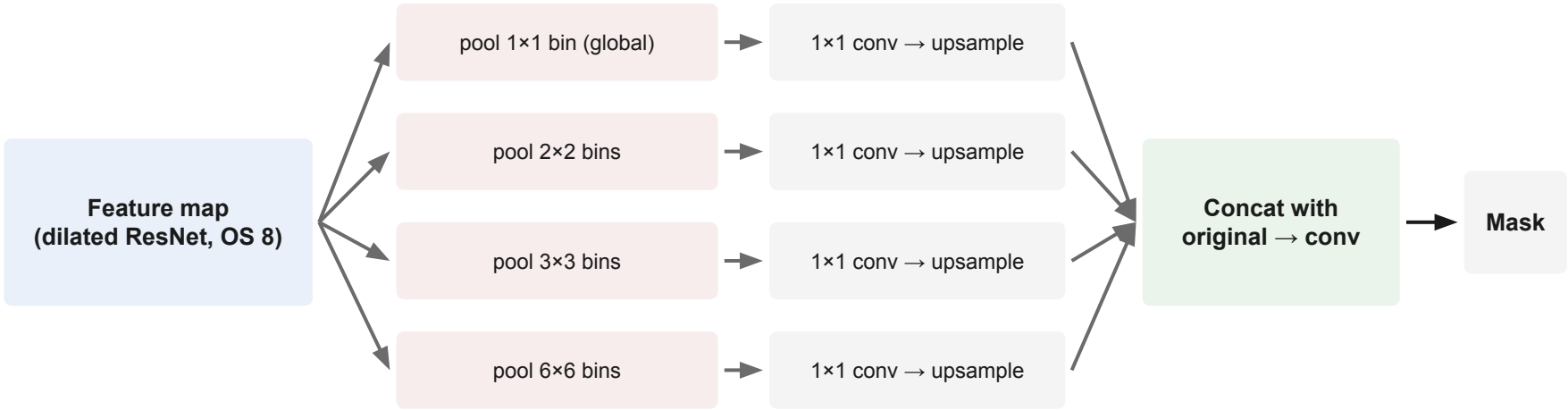
- Multiple receptive fields **in parallel** on the same features
- **Image pooling** adds global context (the "whole scene" branch)
- BatchNorm in every branch (v3)

DeepLab v3+: ASPP meets a light decoder (2018)



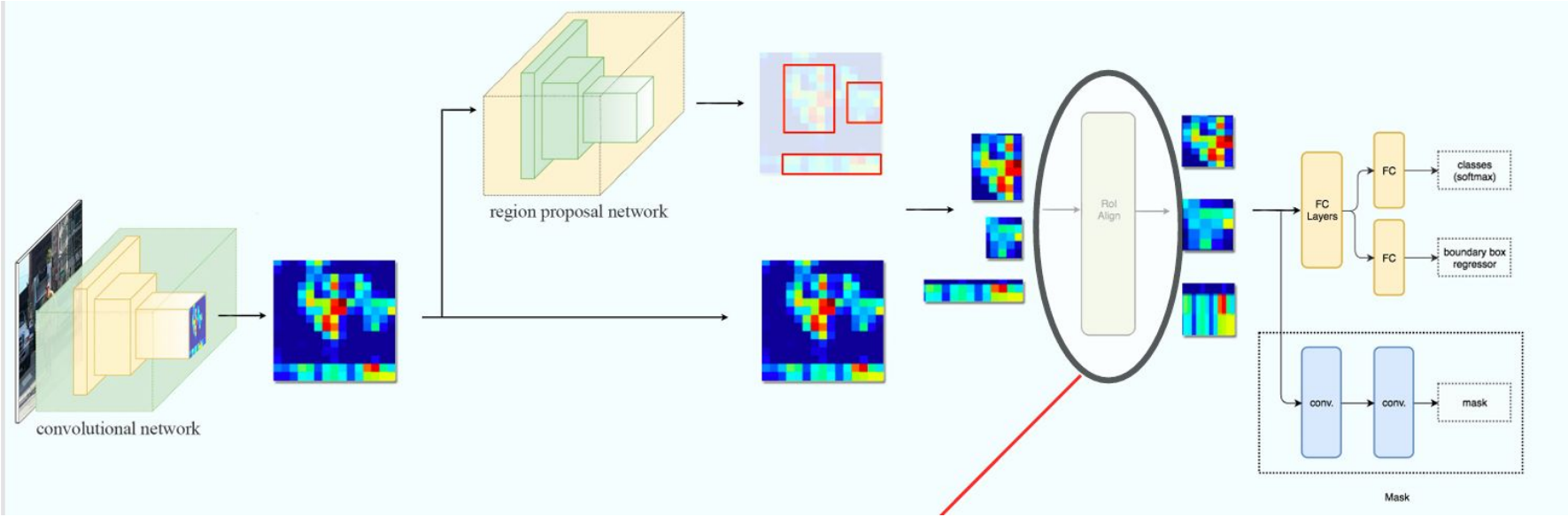
- **Encoder:** atrous backbone + ASPP (semantics, multi-scale context)
- **Decoder:** fuse stride-4 low-level features → sharper boundaries
- **Depthwise-separable atrous convs** (Xception / MobileNet) keep it efficient
- A strong, widely deployed CNN baseline to this day

PSPNet: pyramid pooling for global context



- **Pooling-based** multi-scale context (ASPP is **dilation-based**)
- Global prior fixes context mistakes: boat vs car, pillow vs bed-sheet
- Winner of the ImageNet 2016 scene-parsing challenge

Mask R-CNN

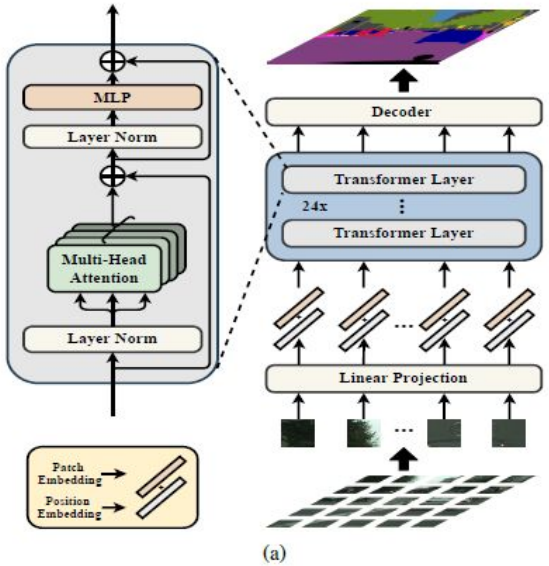


- **Parallel mask branch:** small FCN per RoI, K class-specific 28×28 masks
- **Sigmoid + BCE** per pixel; only the GT class's mask is penalised → mask and class decoupled
- **RoIAlign** instead of RoIPool (next slide) — crucial for pixel accuracy
- L = L_cls + L_box + L_mask; ~5 fps in the original paper

SETR (2021): a pure ViT encoder for segmentation



- **First strong result** with a pure-transformer encoder
- Features are **single-scale at 1/16** — no fine detail from early stages
- **Heavy:** ~318 M parameters for ~50.2 mIoU on ADE20K (as reported in the SegFormer paper)
- Positional embeddings tie it to the training resolution



Thank YOU!